$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Digital Circuits:-

A digital circuit is an electronic circuit that operates on discrete digital signals. These signals exist in two distinct states, typically represented as 0 (low, 0 volts) and 1 (High, 5 volts). The primary purpose of a digital circuit is to process binary information for computing and logic operations.

Components of a digital circuit include:

- Logic Gates: The fundamental building blocks that perform basic logical operations. (AND, OR, NOT).
- Combinational Logic Circuits: Circuits such as multiplexers, encoders, and decoders where the output depends strictly on the current input.
- Sequential Logic Circuits: Circuits including flip-flops, registers, and counters that have memory and depend on both current inputs and past states.

Scanned with CamScanner

Boolean Algebra :-

Boolean Algebra uses variables and operators to describe a logic circuit. It is the mathematical foundation of digital systems.

- **Variable:** A symbol (A-Z, a-z) that represents a logical quantity. A variable can take on a value of either 1 or 0.

- **Complement:** The inverse of a variable. If a given variable is 1 ($A = 1$), its complement is 0 ($\overline{A} = 0$). bar over the letter.

- **Boolean Expression:** A mathematical equation describing the relationship between inputs and outputs. OR A boolean expression is an arrangement of variables and logical operators used to express the operation of a logic circuit.

- **Truth Table:** A truth table lists all input combinations with the corresponding outputs. It is the ultimate proof of a logic circuit expression.

OR

It is a systematic way of listing of values for dependent variable in terms of all possible values of independent variables.

(Formula for all combination = 2^n) n = no of input variables.

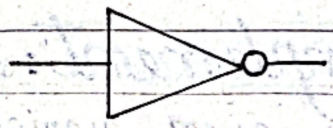
Basic Logic Gates:-

Logic gates are one of the fundamental building blocks of digital systems.

i) The NOT Gate (Inverter):-

A single input gate that produces the exact opposite (complement) of its input.

- Boolean Expression: $X = \overline{A}$

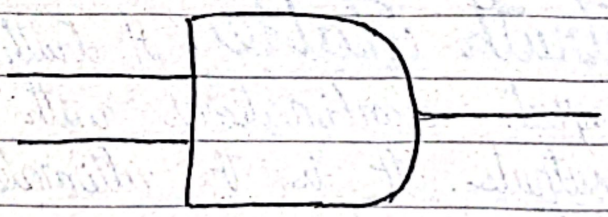


ii) AND Gate:-

A gate that performs logical multiplication. The output is high (1) only if all of its inputs are HIGH (1). If any input is 0, the output is 0.

- Boolean Expression: $X = A \cdot B$

Input		Output
A	B	A·B
0	0	0
1	0	0
0	1	0
1	1	1



Scanned with CamScanner

iii) OR Gate :-

A gate that performs logical addition. The output is high (1) if any of its inputs are high (1). The output is 0 only when all inputs are 0.

- Boolean Expression: $X = A + B$

Boolean Function :-

A mathematical expression formed using binary variables, logical operators and constants (0, 1). It maps the exact relationship between a digital circuit's input and its final output. Denoted as X or F ($F = (A+B) + \bar{C}$).

Laws of Boolean Algebra :-

A set of mathematical rules used to simplify complex boolean expressions.

Simplifying an expression reduces the number of logic gates needed, which minimizes hardware cost, power consumption, and circuit size.

Scanned with CamScanner

1. **Identity Law**: Adding 0 or multiplying by 1 does not change the variable.

$$A + 0 = A$$
$$A \cdot 1 = A$$

2. Dominance Law: Adding 1 forces the output to 1; multiplying by 0 forces the output to 0.

$$A + 1 = 1$$
$$A \cdot 0 = 0$$

3. Idempotent Law: A variable combined with itself results in the same variable.

$$A + A = A \quad ; \quad A \cdot A = A$$

4. Complementary Law: A variable combined with its exact opposite always resolves to a constant.

$$A + \bar{A} = 1 \quad . \quad A \cdot \bar{A} = 0$$

5. Commutative Law: The order in which variables are applied does not matter.

$$A + B = B + A$$
$$A \cdot B = B \cdot A$$

Scanned with CamScanner

7. Associative Law: Grouping of identical operations does not matter.

$$A + (B + C) = (A + B) + C$$
$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

7. Distributive Law: Allows you to multiply across terms or factor variables out.

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$
$$A + (B \cdot C) = (A + B) \cdot (A \cdot C)$$

8. Absorption Law: Shortcut rule that quickly eliminates redundant variables.

$$A + (A \cdot B) = A$$
$$A \cdot (A + B) = A$$

Shortcut: $A + (\bar{A} \cdot B) = A + B$

$A \cdot (\bar{A} + B) = A \cdot B$

9. Involution Law: Inverting a variable twice cancels out the inversion.

$$\bar{\bar{A}} = A$$

Advanced Logic Gates:-

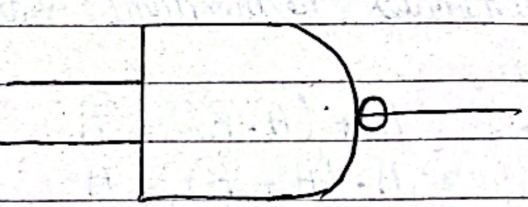
These gates are built upon the foundation of the basic gates.

NAND Gate:-

A universal gate that acts as an inverted AND gate. It outputs low (0) only when all inputs are high (1). When any of the inputs is low, the output will be high.

$$X = \overline{A \cdot B}$$

Input		Output
0	0	1
1	0	1
0	1	1
1	1	0



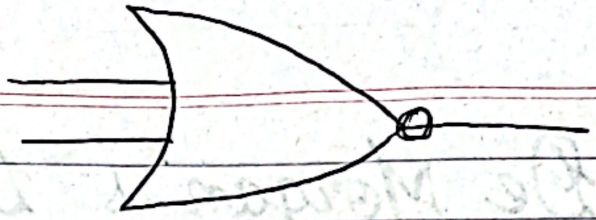
NOR Gate:-

A universal gate that acts as an inverted OR gate. It outputs high (1) only when all inputs are low (0). If any input is 1, the output is 0.

$$X = \overline{A + B}$$

Scanned with CamScanner

Input		Output
0	0	1
1	0	0
0	1	0
1	1	0

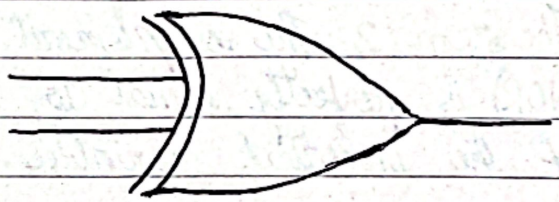


Exclusive-OR Gate (XOR) :-

It outputs high (1) only when the inputs are different. If inputs are identical, it outputs 0.

$$X = A \oplus B = A\bar{B} + \bar{A}B$$

Input		Output
0	0	0
1	0	1
0	1	1
1	1	0

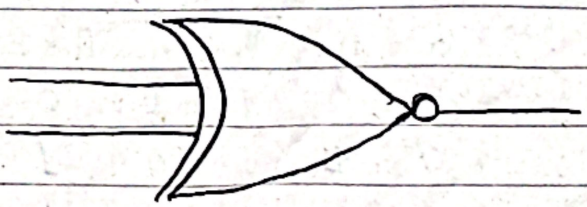


Exclusive-NOR Gate (XNOR) :-

It outputs high (1) only when the inputs are exactly the same. It is the inverse of XOR gate.

$$X = A \odot B = (A \cdot B) + (\bar{A} \cdot \bar{B})$$

Input		Output
0	0	1
1	0	0
0	1	0
1	1	1

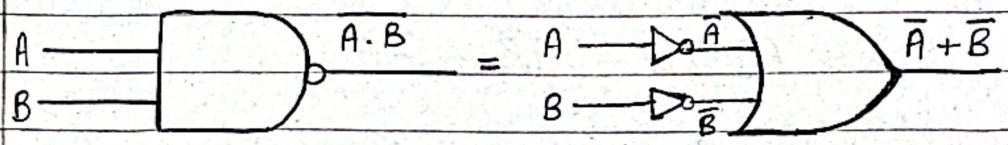


Scanned with CamScanner

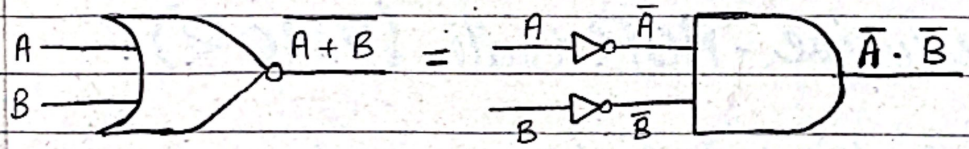
* Break the bar change the sign.

De Morgan's Law :-

- Theorem 1 : The complement of a product equals to the sum of the complements.
- Boolean Expression : $\overline{A \cdot B} = \overline{A} + \overline{B}$



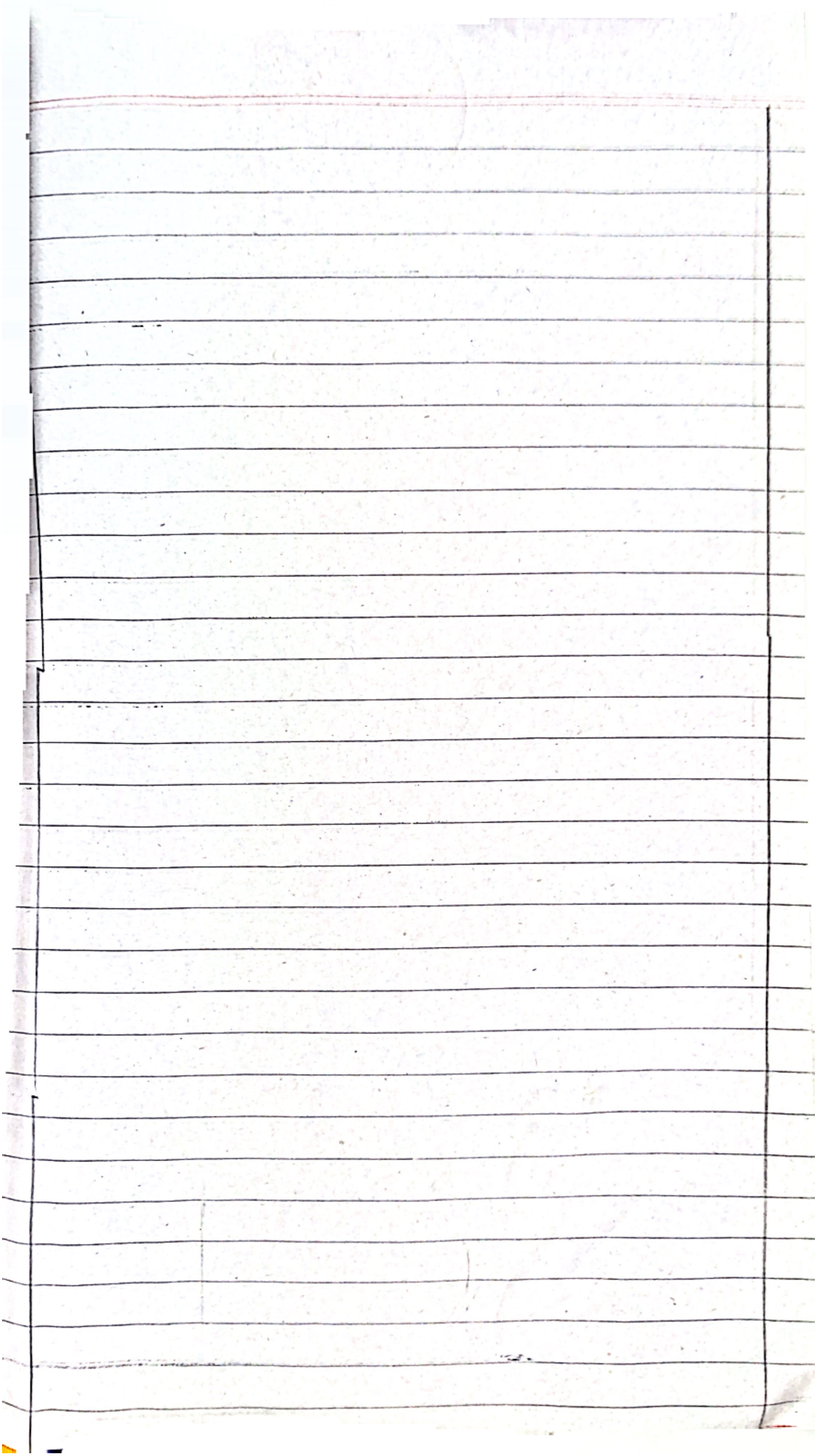
- Theorem 2 : The complement of a logical sum (NOR) is perfectly equal to the logical product of the inverted variables.
- Boolean Expression : $\overline{A+B} = \overline{A} \cdot \overline{B}$



S.no	B	A	$\overline{B}$	$\overline{A}$	A · B	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
	0	0	1	1	0	1	1
	0	1	1	0	0	1	1
	1	0	0	1	0	1	1
	1	1	0	0	1	0	0

S.no	B	A	$\overline{B}$	$\overline{A}$	A + B	$\overline{A+B}$	$\overline{A} \cdot \overline{B}$
	0	0	1	1	0	1	1
	0	1	1	0	1	0	0
	1	0	0	1	1	0	0
	1	1	0	0	1	0	0

Scanned with CamScanner



Scanned with CamScanner

Scanned with CamScanner

Standard forms of Boolean Expressions :-

All boolean expressions can be converted into one of two standard forms : Sum-of-Products (SOP) or Product-of-Sums (POS). Standardization is required to map expressions to a Truth Table or Karnaugh Map.

Domain of a boolean Expression : The set of all variables contained in the expression, either complemented or uncomplemented.

Example : The domain of $F = A\bar{B} + A\bar{C}$ is $\{A, B, C\}$.

A) Sum of Products (SOP) Form :-

- **Definition** : An expression consisting of two or more AND terms (products) that are Ored (summed) together.
- General Format : $AB + BCD + A\bar{C}$
- Implementation : Requires a 2-level AND-OR circuit (a layer of AND gates feeding into one final OR gate.

Scanned with CamScanner

- **Standard SOP Form (Minterms):-**

An SOP form is "Standard" when every variable in the domain appears in every single product term. These complete terms are called minterms.

- Non-Standard Example: $A\bar{B} + C$ (missing C in first term, missing A and B in second term).
- Standard Example: $A\bar{B}C + \bar{A}BC + ABC$ (every term has A, B, C).

★ In SOP $A = 1$ and $\bar{A} = 0$
Example $A\bar{B}C$ is binary 101 (Decimal 5).

B) Product of Sum (POS) Form :-

- Definition: An expression consisting of two or more OR terms (sums) that are ANDed (multiplied) together.
- General Form: $(A+B)(\bar{A}+B+C)(A+\bar{C})$
- Implementation: Requires a two-level OR-AND circuit (a layer of OR gates feeding into one final AND gate).

Scanned with CamScanner

Standard POS Form (Maxterms):

A

POS expression is "standard" when every variable in the domain appears in every single sum term. These complete terms are called Maxterms.

- Non-Standard Example: $(A + \overline{B})(C)$
- Standard Example: $(A + \overline{B} + C)(\overline{A} + B + \overline{C})$

* In POS, a sum term equals 0 only its all variables are 0. $A = 0$, $\overline{A} = 1$
Example: The term $(A + \overline{B} + C)$ is written in binary as 0 1 0.

Scanned with CamScanner

The Karnaugh Map :-

A graphical tool used to simplify Boolean logic expressions systematically, eliminating the need for complex algebric manipulation.

- Developed by Karnaugh in 1953.
- Formula : 2^n ; n is the no of variables.

• Kay-Map Structure & Gray Code :-

The arrangement of cells in a k-map is dictated by Gray Code (00, 01, 11, 10), ensuring that physically adjacent cells differ by only one variable.

• 2-Variable Map : (2^2) → 4 cells

A \ B	0	1
0	0	1
1	2	3

A \ B	0	1
0	$\bar{A}\bar{B}$	$\bar{A}B$
1	$A\bar{B}$	AB

• 3-Variable Map : (2^3) → 8 cells

AB \ C	0	1
00	0	1
01	2	3
11	6	7
10	4	5

AB \ C	0	1
00	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$
01	$\bar{A}B\bar{C}$	$\bar{A}BC$
11	$AB\bar{C}$	ABC
10	$A\bar{B}\bar{C}$	$A\bar{B}C$

Scanned with CamScanner

4-Variable Map: (2^4) → 16 cells

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

AB \ CD	00	01	11	10
00	$\bar{A}\bar{B}\bar{C}\bar{D}$			
01				
11				
10				

Rules of Grouping :-

To achieve the most simplified equation, follow these strict rules when drawing groups around the 1s.

1. Size limit: Groups can only contain a number of cells equal to power of 2 (i.e. 1, 2, 4, 8..).

2. Adjacency: Group strictly vertically or horizontally. Never diagonally.

3. Maximize Size: Always prioritize forming the largest group, even it means overlapping.

4. Minimize Quantity: Form the fewest no of groups necessary to cover every single '1' on the map.

5. Map Rolling (Wrap-around): The top and left edge is considered adjacent to the bottom edge and right edge. Groups can wrap around the boundaries.

Scanned with CamScanner

- **Extracting the Final Equation :-**

Once your groups are drawn, evaluate the variables for each group:

- If a variable remains constant across the entire group, keep the variable otherwise eliminate it.

Scanned with CamScanner

Q Group the 1s in each of the K-maps.

1.

AB \ C	0	1
00	1	
01		1
11	1	1
10		

$\bar{A}\bar{B}\bar{C}$, BC, AB

2.

AB \ C	0	1
00	1	1
01	1	
11		1
10	1	1

AB \ C	0	1
00	1	1
01	1	
11		1
10	1	1

$\bar{B}$, $\bar{A}\bar{C}$, AC

3.

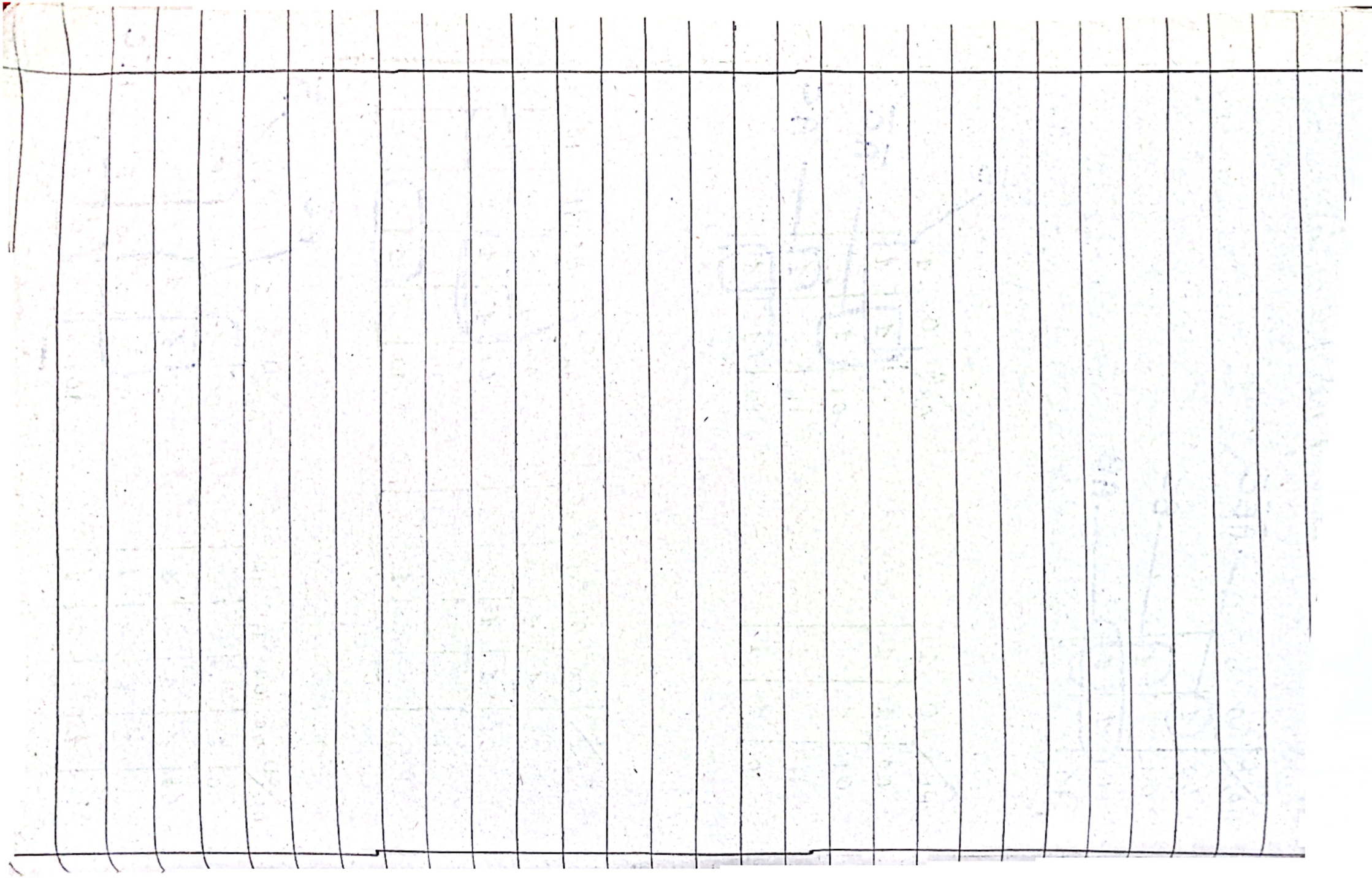
AB \ CD	00	01	11	10
00	1	1		
01	1	1	1	1
11				
10		1	1	

AB \ CD	00	01	11	10
00	1	1		
01	1	1	1	1
11				
10		1	1	

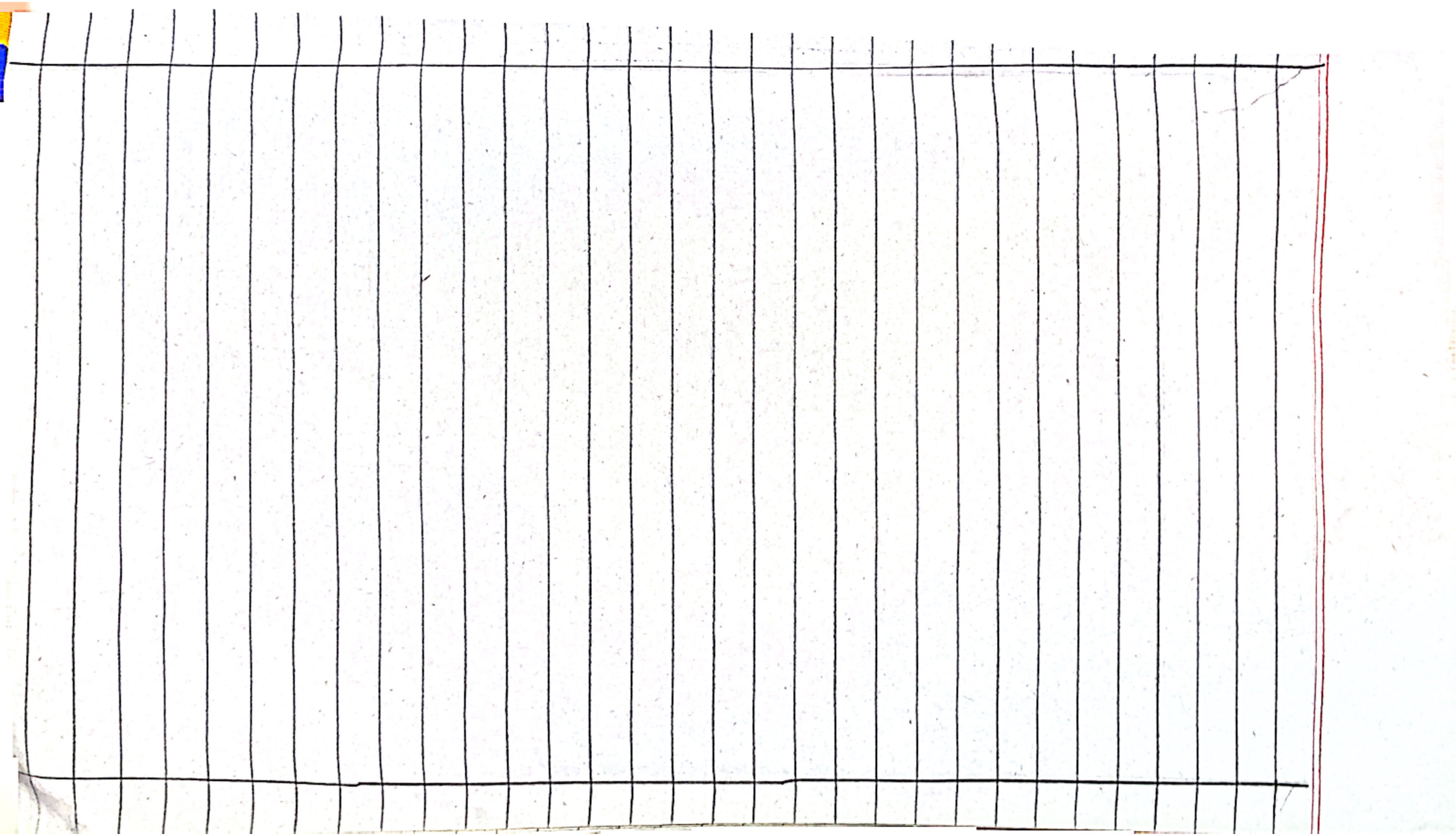
AB \ CD	00	01	11	10
00	1			1
01	1	1		1
11	1	1		1
10	1		1	1

AB \ CD	00	01	11	10
00	1			1
01	1	1		1
11	1	1		1
10	1		1	1

$B\bar{C}$, $\bar{D}$, $A\bar{B}C$



Scanned with CamScanner



Scanned with CamScanner

Scanned with CamScanner

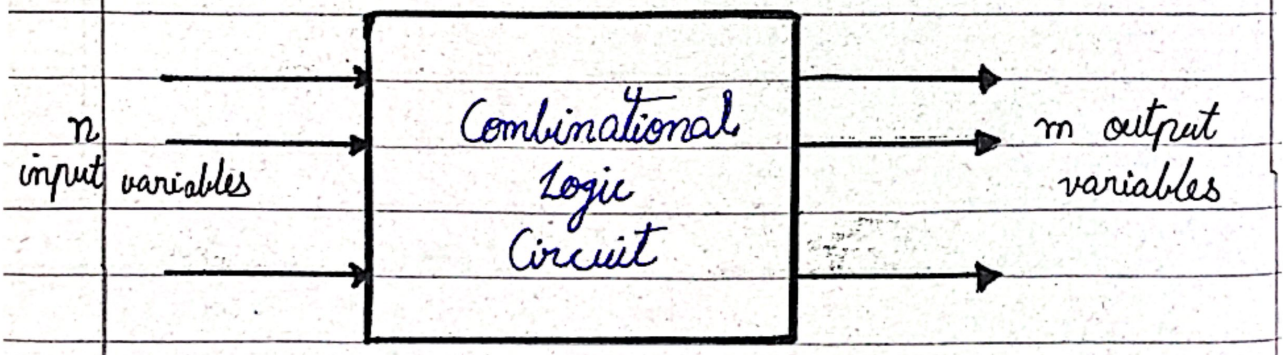
Scanned with CamScanner

Combinational & Sequential Circuits :-

Digital electronics is divided into two broad categories based on whether or not the system requires memory to function.

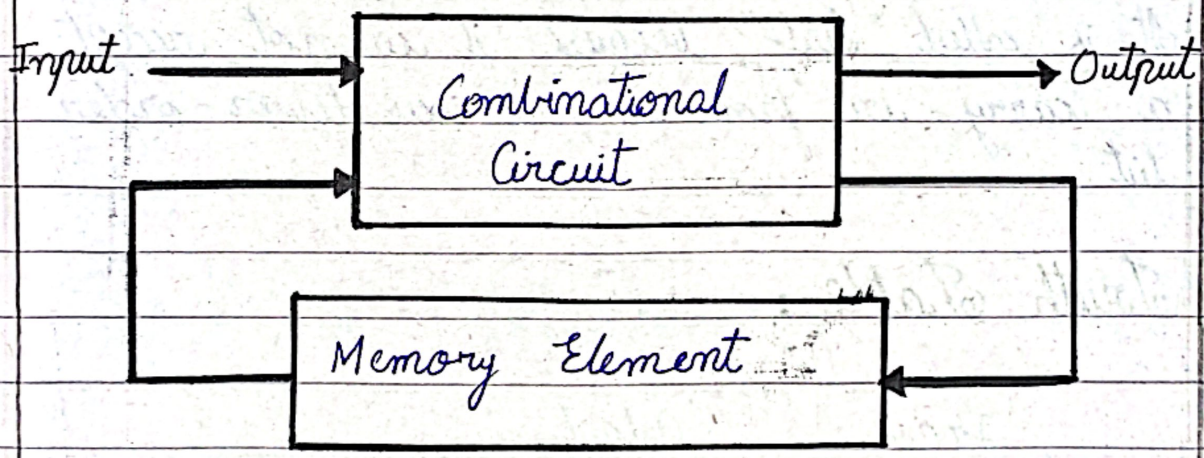
A. Combinational Logic Circuits :-

- Definition: A digital circuit where the present output depends strictly and entirely on the present inputs at the exact moment in time.
- Key Characteristic: It contains no memory elements and no feedback loops.
- Examples: Adders, Subtractors, Multiplexers, Encoders, Decoders.



B. Sequential Logic Circuits :-

- Definition: A digital circuit where the present output depends on the present inputs and the past sequence of inputs (previous state).
- Key Characteristics: It requires memory elements to store past states, and it uses a feedback loop (routing the output back into the input).
- Examples: Latches, Flip-Flops, Registers, Counters, RAM.



Scanned with CamScanner

Adders :-

An adder is a fundamental combinational digital circuit that performs the addition of binary numbers. It forms the core of the Arithmetic Logic Unit (ALU) in modern processors.

A. The Half Adder :-

- Definition : A combinational circuit that adds two single-bit binary inputs (A and B) and produces two outputs : a Sum (S) and a Carry (C).

* It is called "half" because it cannot accept a carry-in from a previous lower-order bit.

- Truth Table :

Input		Output	
A	B	Carry (C)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Scanned with CamScanner

- **Boolean Expressions:**

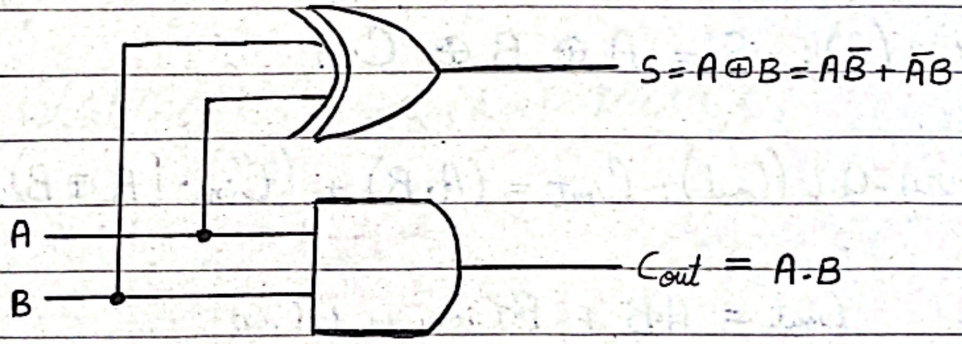
- Sum (S): Output is 1 only when inputs are different.

$$S = A \oplus B \longrightarrow (XOR\ Gate).$$

- Carry (C): Output is 1 only when both inputs are 1.

$$C = A \cdot B \longrightarrow (AND\ Gate)$$

- **Logic Diagram::**



B. The Full Adder :-

- Definition: A combinational circuit that adds three single-bit binary inputs: two data bits (A, B) and a Carry-In (C_{in}) from a previous stage. It produces a Sum (S) and a Carry-Out (C_{out}).

- **Truth Table:**

Scanned with CamScanner

A	B	Cin	Cout :	Sum (S)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

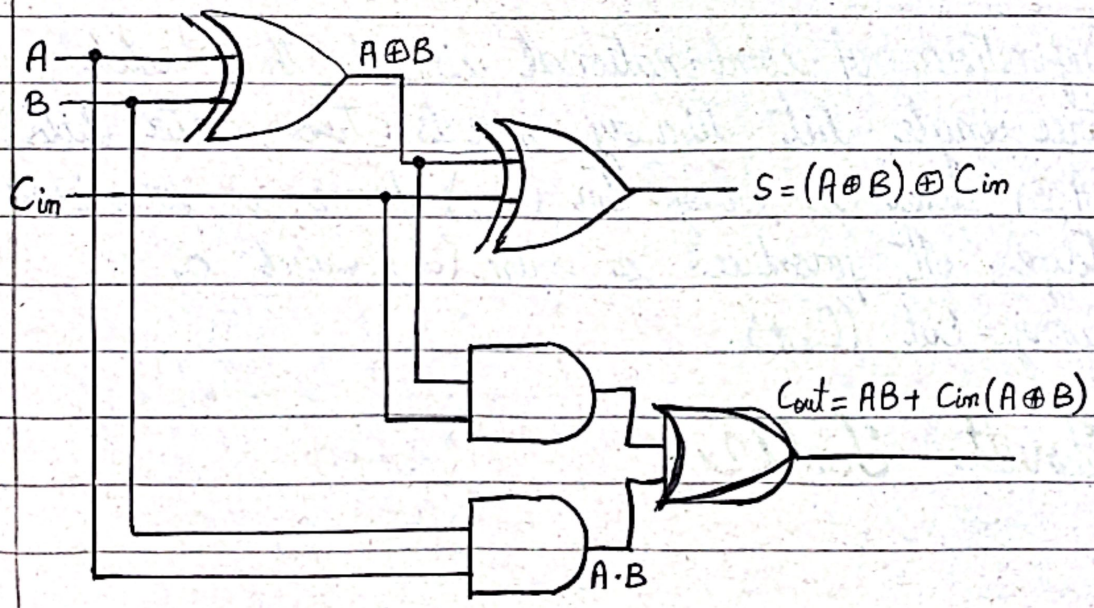
- **Boolean Expressions :**

- Sum (S): $S = A \oplus B \oplus C_{in}$

- Carry-out (C_{out}): $C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$

$$C_{out} = A \cdot B + B \cdot C_{in} + A \cdot C_{in}$$

- **Logic Diagram :**



Subtractor :-

A subtractor is a combinational logic circuit that performs the subtraction of binary numbers.

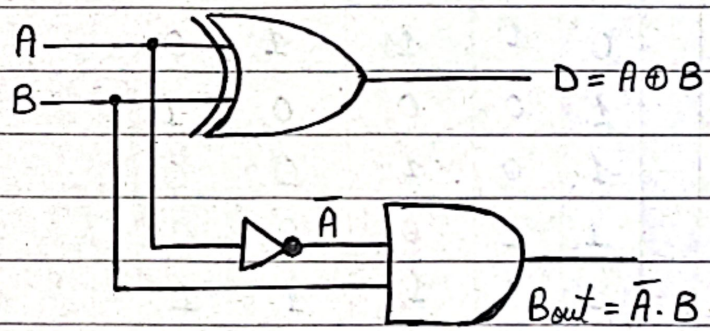
A. The Half Subtractor :-

- **Definition:** A combinational circuit that subtracts two single-bit binary inputs (A - B). It produces two outputs: a Difference (D) and a Borrow-Out (B_{out}). It cannot accept a borrow from previous output.

- **Truth Table:**

A	B	B_{out}	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- **Logic Diagram:**



- **Boolean Expressions:**

- Difference (D): $D = A \oplus B$

- Borrow-Out (B_{out}): $B_{out} = \bar{A} \cdot B$

Scanned with CamScanner

B. The Full Subtractor :-

- **Definition:** A combinational circuit that subtracts ~~two~~ 3 data bits (A - B) and a Borrow-In (Bin) from a previous stage. It produces two outputs: a Difference (D) and a Borrow-Out (Bout).

- **Truth Table:**

Input			Output	
A	B	Bin	Bout	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- **Boolean Expressions:**

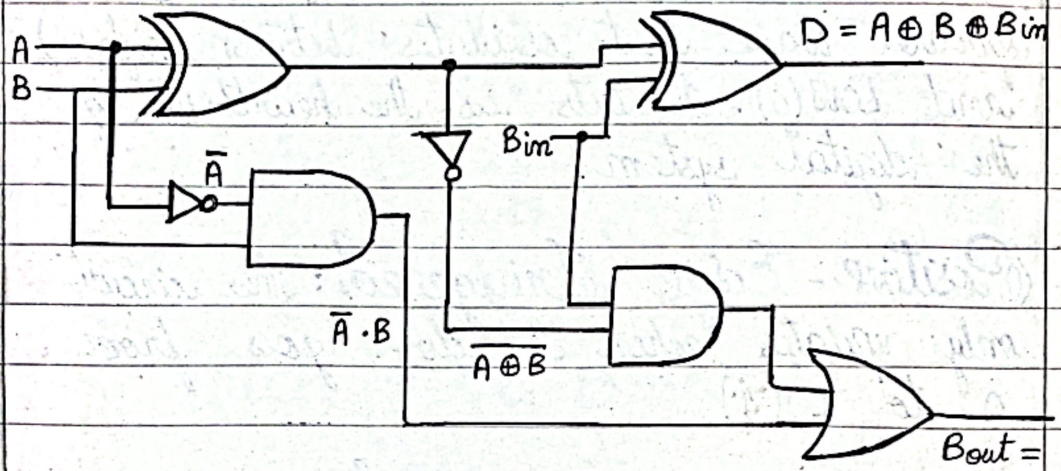
- Difference (D): $D = A \oplus B \oplus Bin$

- Borrow-Out (Bout): $B_{out} = (\overline{A} \cdot B) + (B_{in} \cdot (\overline{A \oplus B}))$

$$B_{out} = \overline{A} \cdot B + \overline{A} \cdot Bin + B \cdot Bin$$

Scanned with CamScanner

- Logic Diagram:



Timing in Digital Circuits:-

Sequential circuits rely heavily on timing to coordinate data flow. There are two categories:

- **Asynchronous Circuits**: The outputs change state immediately whenever any of the inputs change. They do not wait for a coordinating signal. They are faster but very difficult to design without errors.
- **Synchronous Circuits**: The outputs only change state at specific, coordinated moments dictated by a clock signal.

Scanned with CamScanner

The Clock Signal :-

A continuous square wave that oscillates between High (1) and LOW (0). It acts as the heartbeat of the digital system.

- **Positive - Edge Triggered**: The circuit only updates when the clock goes from 0 to 1 (↑)
- **Negative - Edge Triggered**: The circuit only updates when when the clock goes from 1 to 0 (↓)

Latches :-

- **Definition**: A latch is a temporary ~~device~~ storage device (memory element) that has two stable states (bistable). It can store exactly 1 bit of information.
- **Key Characteristics**: Latches are typically level - sensitive (asynchronous), meaning their output reacts immediately as long as the input is held at a certain level.

Scanned with CamScanner

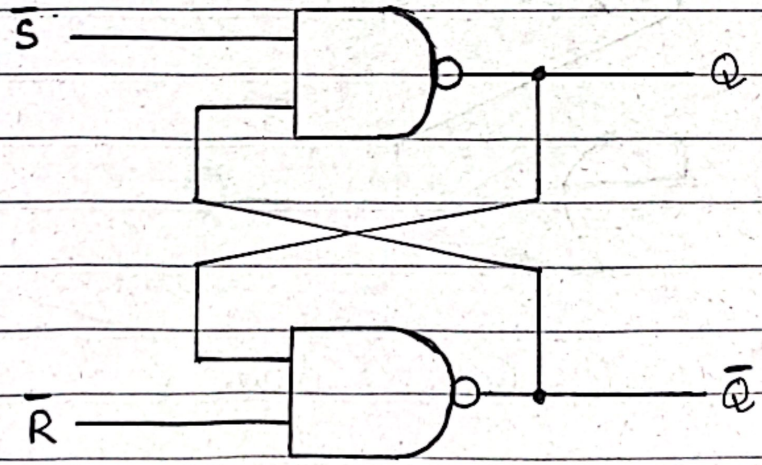
The SR (Set-Reset) Latch :-

- **Definition:** The most fundamental memory circuit. It uses a cross-coupled feedback loop to store a single bit. It is built using either 2 NOR gates (Active-High) or two NAND gates (Active-LOW).

A) SR Latch (NAND-Based) :-

Built using two cross-coupled NAND gates with active-LOW inputs ($\bar{S}$ and $\bar{R}$).

- **Logic Diagram & Truth Table :-**



$\bar{S}$ (set)	$\bar{R}$ (Reset)	Q_{n+1}	State
0	0	X	Invalid
0	1	1	Set
1	0	0	Reset
1	1	Q_n	Hold

Scanned with CamScanner

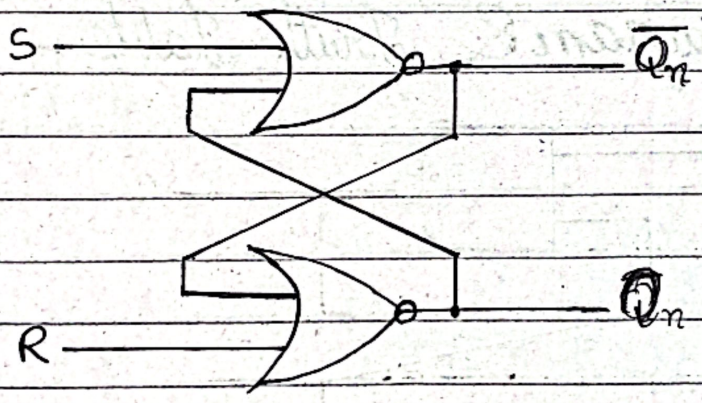
- Characteristic Equation : $Q_{n+1} = S + \bar{R}Q_n$

* Because NAND gates invert logic, the "Hold" state occurs when both inputs are high (1,1), which is the exact opposite of NOR-based latch.

B) SR Latch (NOR - Based) :-

Built using two cross-coupled NOR gates with active-High inputs (S and R).

- Logic Diagram & Truth Table :



S (Set)	R (Reset)	Q_{n+1}	State
0	0	Q_n	Hold
0	1	0	Resel
1	0	1	Set
1	1	X	Invalid

- Characteristic Equation : $Q_{n+1} = S + \bar{R}Q_n$

Scanned with CamScanner

* The Invalid State (1,1) must be avoided because it forces both Q and $\overline{Q}$ to 0, breaking the fundamental rule that the outputs must be opposites.

Flip-Flops :-

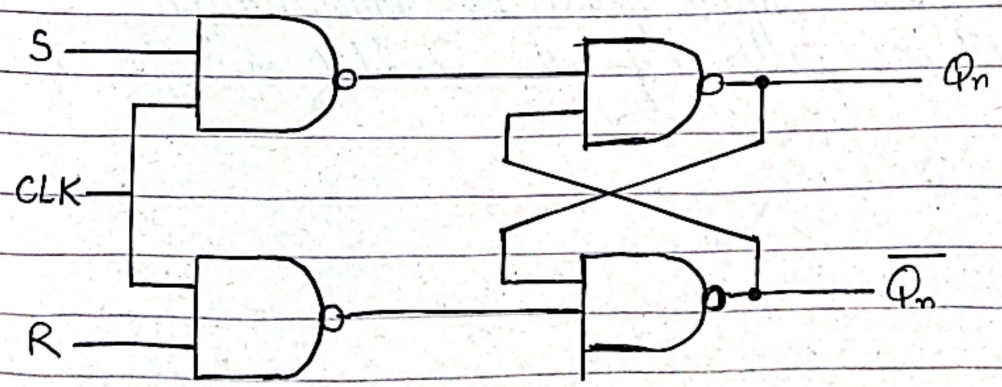
A flip-flop is a synchonous bistable memory device. Unlike latches (which are level-sensitive), flip-flops are edge-triggered meaning their outputs only change state during the precise transition of a clock signal (either rising edge ↑ or falling edge ↓).

A) The SR (Set-Reset) Flip Flop :-

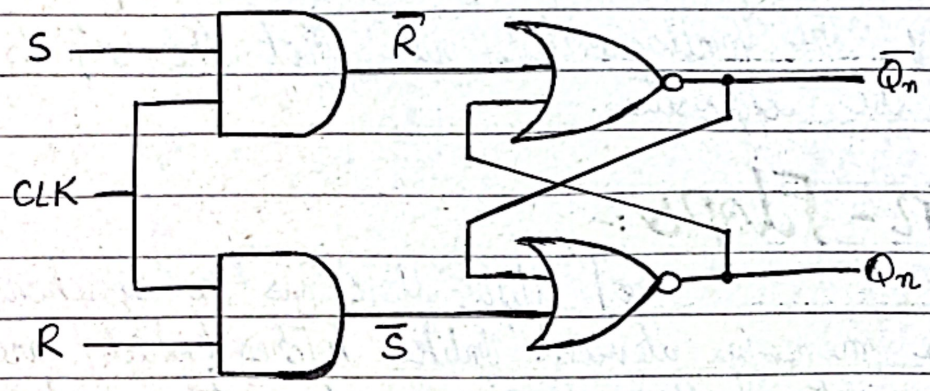
The clocked version of the SR latch.

- Inputs: Set (S), Reset (R), and Clock (CLK).
- Flaw: It still suffers from the invalid state when S = 1 and R = 1.

- **Logic Diagram :-** (NAND-Based)



(NOR-Based)



- **Truth Table :-**

Clock	S (Set)	R (Reset)	Q_{n+1}	State
0	X	X	Q_n	Hold
↑	0	0	Q_n	Hold
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	X	Invalid

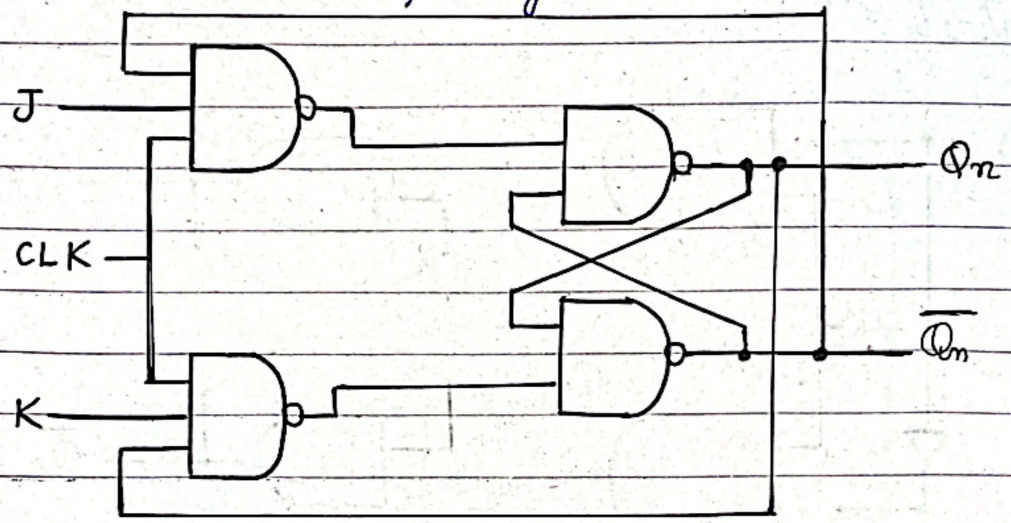
$$Q_{n+1} = S + \overline{R}Q_n$$

★ It provides perfect synchronous timing control, but still suffers from the unpredictable hardware crash of the invalid state (1, 1).

Scanned with CamScanner

B) The JK Flip Flop :-

The most versatile flip-flop. It resolves the undefined state of the SR flip-flop by feeding the outputs back into the input gates.



Clock	J	K	Q_{n+1}	State
0	X	X	Q_n	Hold
↓	0	0	Q_n	Hold
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\overline{Q_n}$	Toggle
↑				

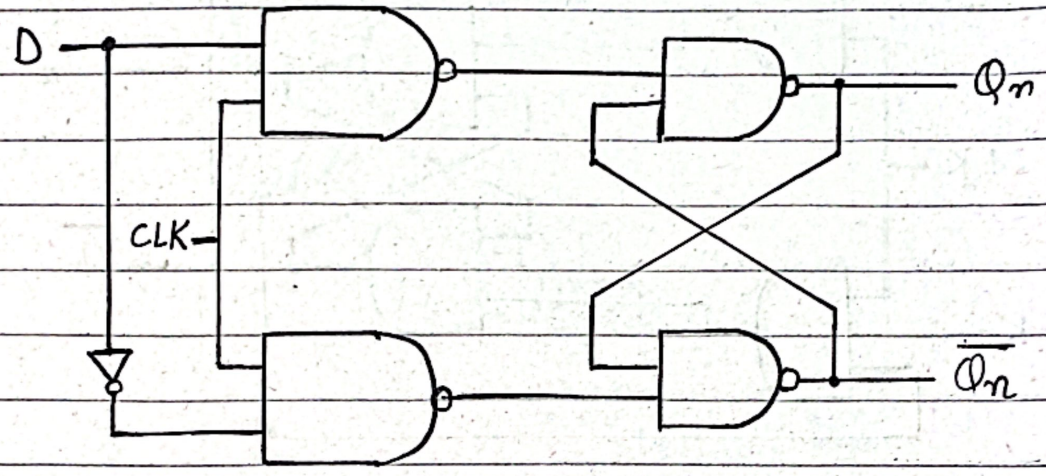
$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

* When J = 1 and K = 1, instead of crashing, the flip-flop safely enters "Toggle Mode" (flipping 1 to 0, or 0 to 1), which is the core mechanism used to build digital counters and clocks.

Scanned with CamScanner

c) The D (Data/Delay) Flip Flop :-

Created by connecting the S and R inputs together through an inverter. It ensures S and R are always opposite, eliminating the invalid state entirely.



Clock	D	Q_{n+1}
0	X	Q_n (Hold)
↑	0	0 (Reset)
↑	1	1 (Set)

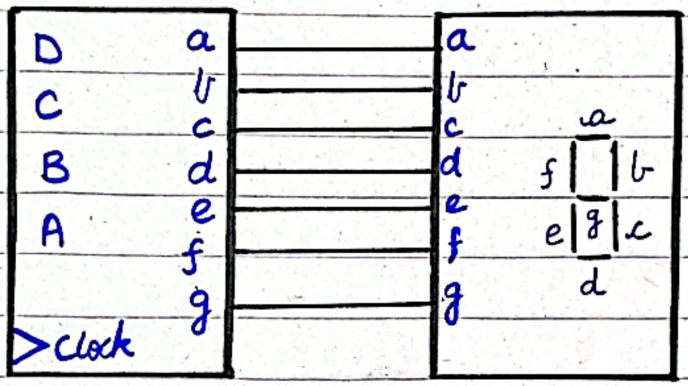
$$Q_{n+1} = D$$

* By enforcing opposite inputs, it completely eliminates the invalid state, making it the standard, flawless building block for CPU, Registers and RAM.

Scanned with CamScanner

BCD to 7-Segment Decoder :-

- Definition : A combinational logic circuit that accepts a 4-bit Binary Coded Decimal (BCD) input and generates 7 specific output signals to illuminate the corresponding decimal digit (0-9) on a 7-segment display.



BCD to 7-Segment Decoder

7-Segment LED Display

Decimal	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

Scanned with CamScanner